

Buildings Built in Minutes - SfM

RBE 549: P2

Pau Alcolea
Department of Robotics
Worcester Polytechnic Institute
palcolea@wpi.edu

Ben Orr
Department of Robotics
Worcester Polytechnic Institute
bcorr@wpi.edu

Abstract—3-Dimensional construction of objects and settings is a subject with a lot of demand in our modern day, specially in the field of robotics, where localization and scanning is used in abundance. The first part of this project, focused on Structure from Motion (SfM), a method of getting 3D models from a series of 2D images, using feature correspondences, estimates of the camera poses and various layers of adjustments with non-linear optimization methods for the reprojection and a final step of bundle adjustments to finalize the pipeline.

I. PROBLEM STATEMENT

In this project we learn how to reconstruct a 3D scene and simultaneously obtain the camera poses of a monocular camera with respect to the given scene. We are using a series of 2D images of the same scene with overlapping features from slightly different angles, attempting to reconstruct the scene with a single camera in motion.

II. PHASE 1: CLASSICAL APPROACH TO THE SFM PROBLEM

The classical approach to 3D modeling from a set of 2D images from slightly different angles begins with the set of corresponding points between images. These point correspondences are based on features, which are used to estimate the camera poses at each of the frames. Using the Fundamental and the Essential matrix, the 3D World Points are obtained, creating the map of the space one image at a time. This map is cleaned up through RANSAC to remove outliers, and the geometric error (looking at the reprojection of the found world points back into the images), refines the camera poses and the world point coordinates in a series of triangulation and PnP. The very end of the classical pipeline uses Bundle Adjustment to refine all the camera poses and 3D points at once in a final step.

A. Feature Matching, Fundamental Matrix and RANSAC

Feature matching is crucial for SfM, as it finds the corresponding points between two images and establishes a geometric constraint that allows us to estimate poses and other information. In this project, the feature matching was done ahead of time and was stored in matching.txt files. Each file corresponded to an image and contained all of the points in other images that had a point in common with the picture in question. SIFT was used, ensuring that the matches are

robust, crucial for the pipeline, as everything relies on good corresponding points between images.

The images in question were taken with the same camera, which was calibrated using using a Radial-Tangential model in MATLAB R2022a's Camera Calibrator Application. The intrinsic camera matrix contained this information, and was also given to as part of the data for this project.

The fundamental matrix F was estimated using a standard RANSAC implementation, as apparent in Algorithm 1. To increase the accuracy of our F estimates, we adopted the Sampson distance as our error metric, as is used in MATLAB's `estimateFundamentalMatrix()` [1], [2]. While the standard algebraic error can be adequate for testing the epipolar constraint (Equation (1)), it has arbitrary scale and therefore doesn't encode geometric relationships. The Sampson distance squares the algebraic error and divides it by its gradient magnitude, returning a normalized difference in squared pixels, i.e. the first order approximation of geometric distance (Equation (2)).

$$x'^T F x = 0 \quad (1)$$

$$d_S(\mathbf{x}, \mathbf{x}', F) = \frac{(\mathbf{x}'^T F \mathbf{x})^2}{(F \mathbf{x})_1^2 + (F \mathbf{x})_2^2 + (F^T \mathbf{x}')_1^2 + (F^T \mathbf{x}')_2^2} \quad (2)$$

B. Estimating Fundamental Matrix

To estimate the fundamental matrix F requires a minimum of 8 non-collinear correspondences. Between two images, 8 pairs of points (like eq. (3)) will give us 8 degrees of freedom, which will be enough to solve for our 3x3 matrix F because one of the degrees of freedom is taken up by the scale.

The fundamental matrix encodes the epipolar geometry between two images, and it tells that x' must lie on the epipolar line Fx . That epipolar line on the second image corresponds to the ray that belongs to the point x on the first. This leads us to the epipolar constraint $x'^T F x = 0$, which when expanded out with eq. (3) and eq. (4) gets us eq. (5).

$$\mathbf{x}_i = \begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix}, \quad \mathbf{x}'_i = \begin{bmatrix} u'_i \\ v'_i \\ 1 \end{bmatrix} \quad (3)$$



Fig. 1. Dataset provided for Phase 1

Algorithm 1 RANSAC for Fundamental Matrix Estimation

Require: Correspondences $\mathcal{C} = \{(\mathbf{x}_i, \mathbf{x}'_i)\}_{i=1}^N$

Ensure: Fundamental Matrix F

```

1:  $M_{\text{best}} \leftarrow \emptyset$ 
2:  $n_{\text{best}} \leftarrow 0$ 
3: for  $k = 1$  to  $K$  do
4:   Randomly sample a minimal set  $\mathcal{S}$  of 8 correspondences from  $\mathcal{C}$ 
5:   Estimate  $F$  from  $\mathcal{S}$  (normalized 8-point method with rank-2 enforcement)
6:    $M \leftarrow \emptyset$ 
7:   for each correspondence  $(\mathbf{x}, \mathbf{x}') \in \mathcal{C}$  do
8:     Compute Sampson distance  $d_S(\mathbf{x}, \mathbf{x}', F)$ 
9:     if  $d_S(\mathbf{x}, \mathbf{x}', F) < \epsilon$  then
10:      Add  $(\mathbf{x}, \mathbf{x}')$  to  $M$ 
11:     end if
12:   end for
13:   if  $|M| > n_{\text{best}}$  then
14:      $M_{\text{best}} \leftarrow M$ 
15:      $n_{\text{best}} \leftarrow |M|$ 
16:   end if
17: end for
18: Estimate  $F$  from  $M_{\text{best}}$  return  $F$ 

```

$$\mathbf{f} = [f_{11} \ f_{12} \ f_{13} \ f_{21} \ f_{22} \ f_{23} \ f_{31} \ f_{32} \ f_{33}]^T \quad (4)$$

$$\begin{bmatrix} x_i x'_i & x_i y'_i & x_i & y_i x'_i & y_i y'_i & y_i & x'_i & y'_i & 1 \end{bmatrix} \mathbf{f} = 0 \quad (5)$$

This setup allows us to use SVD to solve the equation with our 8 correspondences. It is important to afterwards establish the Rank 2 Constraint (eq. (9)) to make sure that all of the epipolar lines intersect at a point. This crucial step ensures that the estimated camera pose (which uses the Fundamental Matrix), is accurate and that the error is not carried over to all of the remaining steps of the framework. The way to do this is to make the smallest singular value $s_3 = 0$. Keeping all three singular values like in eq. (8) means there's no null space and no epipole.

$$A\mathbf{f} = 0, \quad A \in \mathbb{R}^{8 \times 9} \quad (6)$$

$$A = U\Sigma V^T, \quad \mathbf{f} = \underset{\|\mathbf{f}\|_2=1}{\operatorname{argmin}} \|A\mathbf{f}\|_2 \quad (7)$$

$$F = U_F \operatorname{diag}(\sigma_1, \sigma_2, \sigma_3) V_F^T, \quad \sigma_1 \geq \sigma_2 \geq \sigma_3 \quad (8)$$

$$F_{\text{rank2}} = U_F \operatorname{diag}(\sigma_1, \sigma_2, 0) V_F^T \quad (9)$$

Like stated in the previous section, the fundamental matrix is then taken through RANSAC to make sure that there are no outliers in the 8 chosen correspondences negatively affecting F .

C. Estimate Essential Matrix from Fundamental Matrix

The essential matrix E , which assumes that the cameras follow the pinhole model, takes advantage of the the computed F to find the relative camera poses between two images, eq. (10). Instead of relating two points like F , it relates two cameras with the eq. (11).

$$E = K^T F K \quad (10)$$

$$\mathbf{x}'^T E \mathbf{x} = 0 \quad (11)$$

D. Estimate Camera Pose from Essential Matrix

Once the essential matrix is calculated, algorithm algorithm 2 was used to extract the possible camera poses for the second image. Because the F and E deal with relative poses, the first image is set to the origin, and the pose for the camera of the second image is "built" off that. The pose consists of $P = K[R|t]$, so the goal of the algorithm is to get the translation candidates and the Rotation matrix.

Singular Value Decomposition is used on E , getting us two possible translations: $T = u_2$ or $T = u_3$ because the epipole will lie on the plane. The left nullspace is in the second image and $T^T \times E = 0$. With that information, we go for the calculation of the rotation matrix, which we aim go to acquire with eq. (12), and eq. (13)

$$E = [t]_X R \quad (12)$$

$$T_X = u_1 \times u_2 = U \begin{bmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} U^T \quad (13)$$

Algorithm 2 Extracting Camera Poses from E

Require: Essential matrix $E \in \mathbb{R}^{3 \times 3}$ **Ensure:** Four possible camera poses $\{[R_i | t_i]\}_{i=1}^4$

1: Compute Singular Value Decomposition:

$$E = U\Sigma V^T$$

2: Define matrix

$$W = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

3: Compute two possible rotations:

$$R_1 = UWV^T, \quad R_2 = UW^TV^T$$

4: Extract translation direction (up to scale):

$$t = \mathbf{u}_3$$

where $\mathbf{u}_3$ is the third column of U

5: Define translation candidates:

$$t_1 = t, \quad t_2 = -t$$

6: Initialize pose set $\mathcal{P} \leftarrow \emptyset$ 7: **for** each $R \in \{R_1, R_2\}$ **do**8: **for** each $t \in \{t_1, t_2\}$ **do**9: **if** $\det(R) < 0$ **then**10: $R \leftarrow -R$ 11: $t \leftarrow -t$ 12: **end if**13: Append $[R | t]$ to $\mathcal{P}$ 14: **end for**15: **end for****return** $\mathcal{P}$

In the end we are left with two possible translation vector and two possible rotation matrices, which ends in 4 possible camera poses (one per combination of t and R , which have to be disambiguated. This is done with the Chirality Constraint, which looks at the points relate to each of the camera poses. fig. 2 shows the world points and how they look different for each of the possible poses.

E. Triangulation Check for Chirality Condition

To disambiguate the poses, the Chirality Constraint is used, which looks at all of the world points in respect to each of the possible camera poses and then selects the one in which most points are in the positive Z direction. This is accomplished with triangulation. Before making any comparisons between world points and poses, we need to reconstruct said points from projections with a method called triangulation. This uses two corresponding image projections from two different cameras and sees where their rays intersect, getting the world point X . We first accomplish this using linear triangulation and the following algorithm 3.

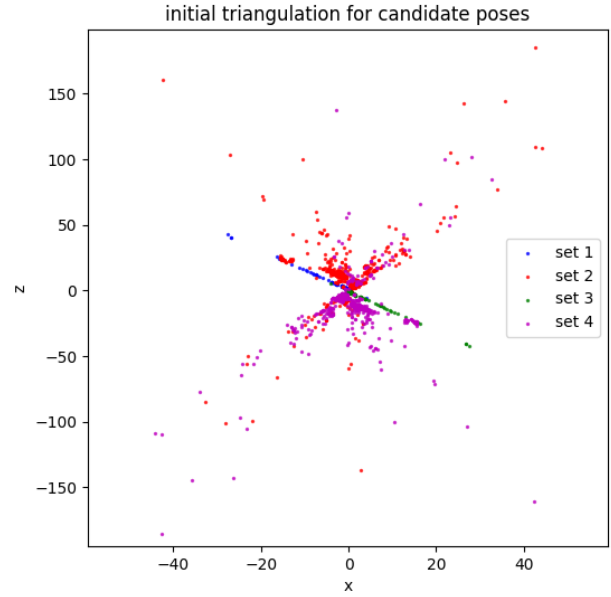


Fig. 2. The four candidate poses extracted from E

Algorithm 3 Pose Disambiguation with Linear Triangulation

Require: Candidate poses $\{[R_i | t_i]\}_{i=1}^4$, intrinsic matrix K , correspondences $\{(\mathbf{x}_j, \mathbf{x}'_j)\}_{j=1}^N$ **Ensure:** Correct camera pose $[R^* | t^*]$ and triangulated 3D points $\{\mathbf{X}_j^*\}$

1: Define reference pose:

$$P_1 = K[I | 0]$$

2: **for** each candidate pose $[R_i | t_i]$ **do**

3: Compute projection matrix:

$$P_2 = K[R_i | t_i]$$

4: Triangulate all correspondences:

$$\mathbf{X}_j^{(i)} = \text{LinearTriangulation}(P_1, P_2, \mathbf{x}_j, \mathbf{x}'_j)$$

5: Count points satisfying chirality:

$$Z_{1j} > 0 \quad \text{and} \quad Z_{2j} > 0$$

where

$$Z_{2j} = \mathbf{r}_3^T \mathbf{X}_j^{(i)} + t_3$$

6: **end for**

7: Select pose with maximum number of valid points

return Best pose, corresponding triangulated points

1) *Linear Triangulation:* Linear triangulation estimates the position of a 3D world point given its projections in various images and the camera poses for them. Each 2D point x_i in image i defines a ray through the camera center, following the pinhole camera model eq. (14). These rays are supposed to

Linear Reprojection of 3D points for image 2

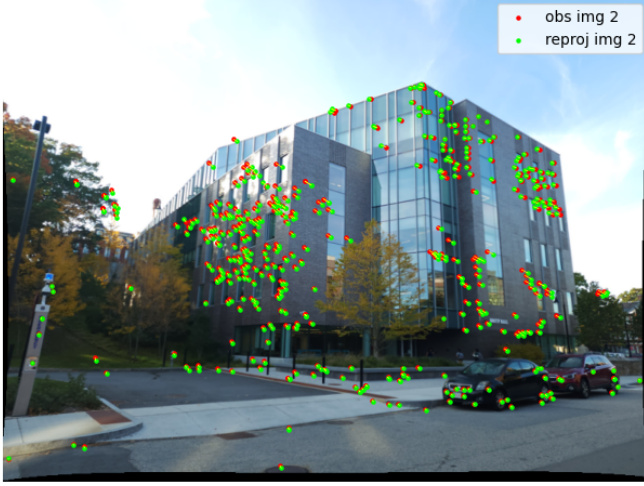


Fig. 3. Reprojected points after linear triangulation for image 2

intersect at the 3D point X , but due to noise they don't exactly. This is solved by using a linear system like in eq. (15), where the cross-product term enforces that the new point X is on the ray corresponding to both projected rays. The system $AX = 0$ is solved with SVD, which gets us X as seen in the last section. To solve this, a linear system is constructed as shown in eq. (16). Figure 5 shows an example of triangulated points using this method for images 1 and 2. Figure 3 shows the reprojected points after linear triangulation for image 2. Other examples can be found in the appendix.

$$\lambda \begin{bmatrix} x \\ 1 \end{bmatrix} = P \begin{bmatrix} X \\ 1 \end{bmatrix} \quad P = K[R \mid t] \quad (14)$$

$$AX = \begin{bmatrix} \begin{bmatrix} x_1 \\ 1 \end{bmatrix} \times P_1 \\ \begin{bmatrix} x_2 \\ 1 \end{bmatrix} \times P_2 \\ \vdots \\ \begin{bmatrix} x_N \\ 1 \end{bmatrix} \times P_N \end{bmatrix} \begin{bmatrix} X \\ 1 \end{bmatrix} = 0 \quad (15)$$

$$A = U\Sigma V^T \quad \mathbf{X} = \underset{\|\mathbf{X}\|=1}{\operatorname{argmin}} \|A\mathbf{X}\| \quad (16)$$

2) *Non-Linear Triangulation*: Non-linear triangulation does not replace its linear counterpart, instead, it builds from it and refines the 3D points it created by explicitly minimizing the reprojection error in image space. Each 3D point X is first projected into image coordinates using the camera's projection matrix as shown in eq. (17), and then converted to pixel coordinates by dehomogenization to produce $(\hat{u}, \hat{v})$. The reprojection error $r(X)$, defined in eq. (19), represents the difference between the projected point from X and the observed image point across all views. We used least squares

Nonlinear Reprojection of 3D points for image 2

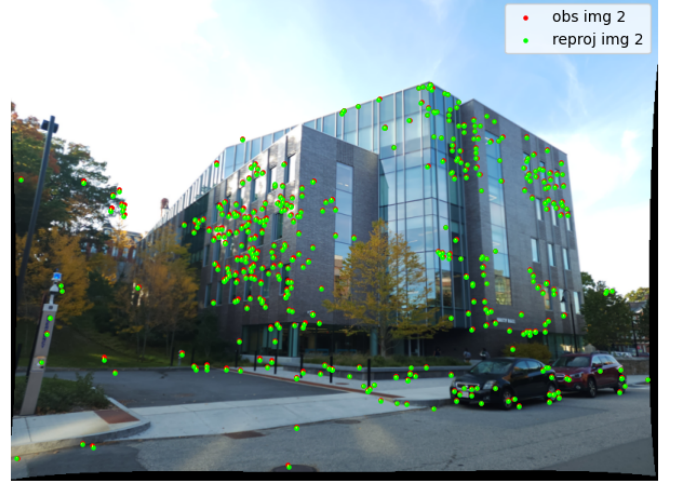


Fig. 4. Reprojected points after nonlinear triangulation for image 2

optimization to get the optimal 3D point X^* as seen in eq. (20). This refinement makes the 3D reconstruction more accurate than the linear triangulation, especially when there's noise in the system and the camera poses estimated are not perfect.

An example of reprojected points after non-linear triangulation can be found in Figure 4. Note the decreased visibility of red dots (observed points) in the image compared to Figure 3, suggesting the success of our implementation.

$$P\mathbf{X} = \begin{bmatrix} \tilde{x} \\ \tilde{y} \\ \tilde{z} \end{bmatrix}, \quad P = K[R \mid t] \in \mathbb{R}^{3 \times 4} \quad \mathbf{X} = \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \quad (17)$$

$$\pi(P, \mathbf{X}) = P\mathbf{X} \frac{1}{\tilde{z}} = \begin{bmatrix} \tilde{x}/\tilde{z} \\ \tilde{y}/\tilde{z} \end{bmatrix} = \begin{bmatrix} \hat{u} \\ \hat{v} \end{bmatrix} \quad (18)$$

$$\mathbf{r}(\mathbf{X}) = \begin{bmatrix} \mathbf{x}_1 - \pi(P_1, \mathbf{X}) \\ \mathbf{x}_2 - \pi(P_2, \mathbf{X}) \end{bmatrix} = \begin{bmatrix} u_1 - \hat{u}_1(\mathbf{X}) \\ v_1 - \hat{v}_1(\mathbf{X}) \\ u_2 - \hat{u}_2(\mathbf{X}) \\ v_2 - \hat{v}_2(\mathbf{X}) \end{bmatrix} \in \mathbb{R}^4 \quad (19)$$

$$\mathbf{X}^* = \underset{\mathbf{X} \in \mathbb{R}^3}{\operatorname{argmin}} \|\mathbf{r}(\mathbf{X})\|_2^2 \quad (20)$$

F. Perspective-n-Points

The previous steps have given us a set of good world points based on two images, which have been refined through non-linear triangulation, but continuing this method for more cameras is not viable. Repeating these previous steps with an already calculated image and the new view would be a less robust strategy, and the noise would accumulate, instead, Perspective-n-Points uses all of the previously triangulated

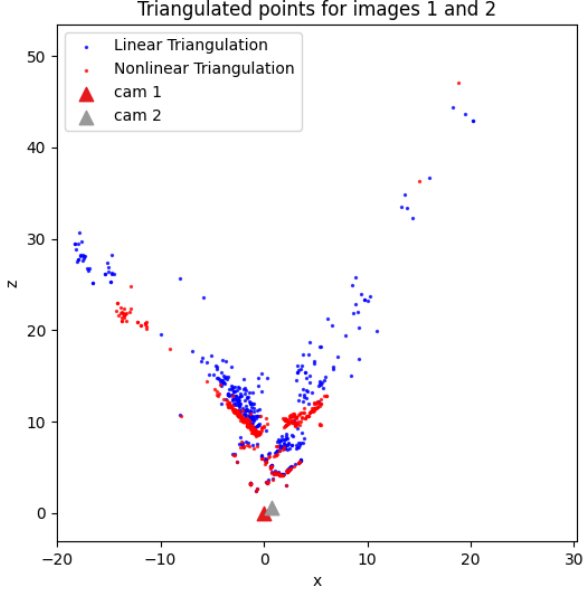


Fig. 5. Top-down view of linear and nonlinear triangulated points from the base pair (images 1-2)

world points to make sure the new estimate for the camera pose is more accurate.

1) *Linear Camera Pose Estimation (PnP)*: Linear PnP is in a RANSAC algorithm to estimate the pose of a camera given a set of 3D points and their corresponding 2D projections in the image. The estimated pose is calculated by making a linear system out of each correspondence and solving it with SVD, as seen in algorithm 4. The singular value decomposition minimizes the algebraic error and provides a projection matrix from which the rotation matrix and translation vector have to be corrected because the linear least square solve doesn't enforce orthogonality of R , as it is not geometric. For this process, six correspondences are required because the system needs to solve 12 degrees of freedom (9 from the rotation and 3 from the translation), and each correspondence brings two equations.

2) *PnP RANSAC*: Like fundamental matrix estimation (Section II-A), linear PnP was implemented inside of RANSAC. The error metric for finding inliers can be found in Equation (21).

$$\hat{\mathbf{x}}_i = \Pi(P\tilde{\mathbf{X}}_i), \quad e_i = \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|_2^2, \quad \text{inlier if } e_i < \epsilon. \quad (21)$$

An example of 3D points, reprojected back to the image plane, is found in Figure 7. Yellow lines in the image connect the observed (ground truth) points and their reprojected counterparts.

3) *Nonlinear PnP*: Like for triangulation, the linear Perspective-n-Point solution can be refined using nonlinear

Algorithm 4 Linear PnP

Require: N correspondences $\{(\mathbf{X}_i, \mathbf{x}_i)\}$ with $\mathbf{X}_i \in \mathbb{R}^3$, $\mathbf{x}_i = (u_i, v_i)^T$, intrinsics K

Ensure: Pose (R, t)

- 1: Form homogeneous points $\tilde{\mathbf{X}}_i = [X_i \ Y_i \ Z_i \ 1]^T$, $\tilde{\mathbf{x}}_i = [u_i \ v_i \ 1]^T$
 - 2: Build a linear system $A\mathbf{p} = 0$ from the constraint $\tilde{\mathbf{x}}_i \times (P\tilde{\mathbf{X}}_i) = \mathbf{0}$, where $\mathbf{p} = \text{vec}(P)$ and $P \in \mathbb{R}^{3 \times 4}$
 - 3: Solve $A\mathbf{p} = 0$ via SVD and reshape $\mathbf{p}$ into P
 - 4: Let $P = [P_{3 \times 3} \ \mathbf{p}_4]$ and compute $M = K^{-1}P_{3 \times 3}$
 - 5: Project M to the closest rotation by SVD: if $M = U\Sigma V^T$, set $R \leftarrow UV^T$
 - 6: Set scale $s \leftarrow \Sigma_{11}$ and translation $t \leftarrow \frac{1}{s}K^{-1}\mathbf{p}_4$
 - 7: **if** $\det(R) < 0$ **then**
 - 8: $R \leftarrow -R, \quad t \leftarrow -t$
 - 9: **end if**
 - return** (R, t)
-

Linear PnP Reprojection of 3D points for image 4

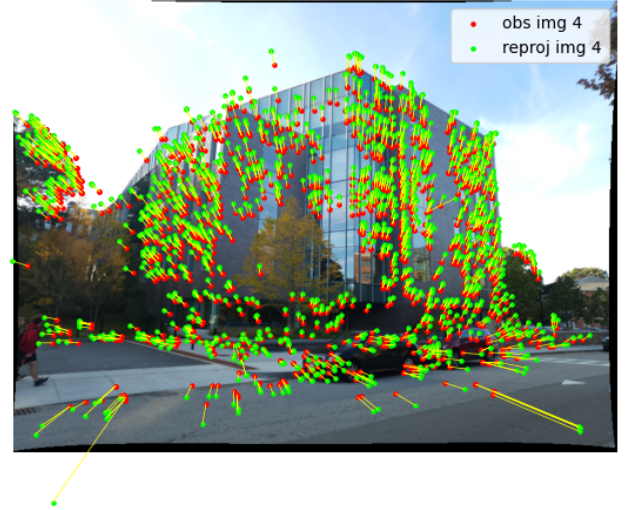


Fig. 6. Reprojected points after linear PnP for image 4

PnP, which minimizes the geometric reprojection error between measured 2D points and projected 3D points (eq. (25)). In this optimization the rotation of each camera pose is encoded as a quaternion $\mathbf{q}$ (eq. (22)), which provides a compact and more efficient representation for optimization. We used nonlinear least squares to optimize the full set of pose parameters $\boldsymbol{\theta} = [\mathbf{t}; \mathbf{q}]$ (eq. (26)–eq. (27)). The difference between this formulation and that for nonlinear triangulation is that the parameters for PnP describe the camera pose as opposed to the 3D point positions. The optimization returns a refined rotation R^* and translation $\mathbf{t}^*$ that minimize the reprojection error, which makes the camera poses more accurate.

$$R(\mathbf{q}) \in SO(3), \quad \mathbf{q} \in \mathbb{R}^4 \quad (22)$$

Nonlinear PnP Reprojection of 3D points for image 4

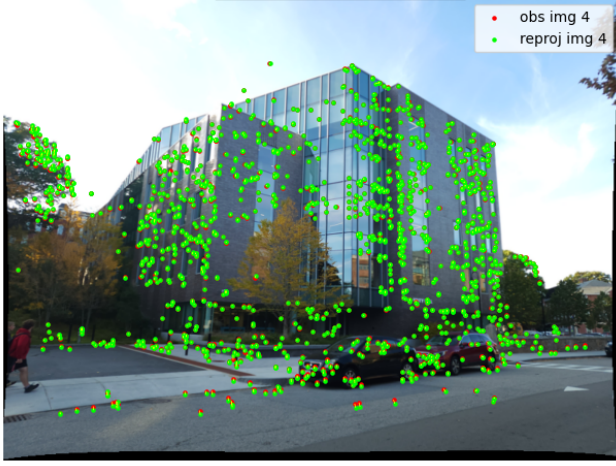


Fig. 7. Reprojected points after nonlinear PnP for image 4

$$\theta = \begin{bmatrix} \mathbf{t} \\ \mathbf{q} \end{bmatrix} \in \mathbb{R}^7, \quad \mathbf{t} \in \mathbb{R}^3 \quad (23)$$

$$\tilde{\mathbf{x}}_i = P(\theta) \tilde{\mathbf{X}}_i = \begin{bmatrix} \tilde{x}_i \\ \tilde{y}_i \\ \tilde{z}_i \end{bmatrix}, \quad \hat{\mathbf{x}}_i = \begin{bmatrix} \hat{u}_i \\ \hat{v}_i \end{bmatrix} = \begin{bmatrix} \tilde{x}_i / \tilde{z}_i \\ \tilde{y}_i / \tilde{z}_i \end{bmatrix} \quad (24)$$

$$\mathbf{r}(\theta) = \begin{bmatrix} \mathbf{r}_1(\theta) \\ \vdots \\ \mathbf{r}_N(\theta) \end{bmatrix} = \begin{bmatrix} u_1 - \hat{u}_1 \\ v_1 - \hat{v}_1 \\ \vdots \\ u_N - \hat{u}_N \\ v_N - \hat{v}_N \end{bmatrix} \in \mathbb{R}^{2N} \quad (25)$$

$$\theta^* = \arg \min_{\theta} \|\mathbf{r}(\theta)\|_2^2 \quad (26)$$

$$\mathbf{q}^* = \frac{\theta_{4:7}^*}{\|\theta_{4:7}^*\|}, \quad R^* = R(\mathbf{q}^*), \quad \mathbf{t}^* = \theta_{1:3}^* \quad (27)$$

After the camera poses have been refined, new points are added to the map by triangulating correspondences between the new camera and 2D points in the already included frames. PnP makes this addition of points more consistent with the existing map, making the SfM reconstruction more robust. The map after adding all 5 cameras with this method can be seen in Figure 8. Reprojected 3D points are visualized in Figure 7 (compare with the linear equivalent in Figure 6).

G. Bundle Adjustment

The final part of the SfM framework is to make a final refinement of all of the points and poses by passing everything into an optimization function. This minimization observes how variations in X or in the camera pose affect the geometric error. This step looks at the relationship between every camera pose and each world point X through a Visibility Matrix.

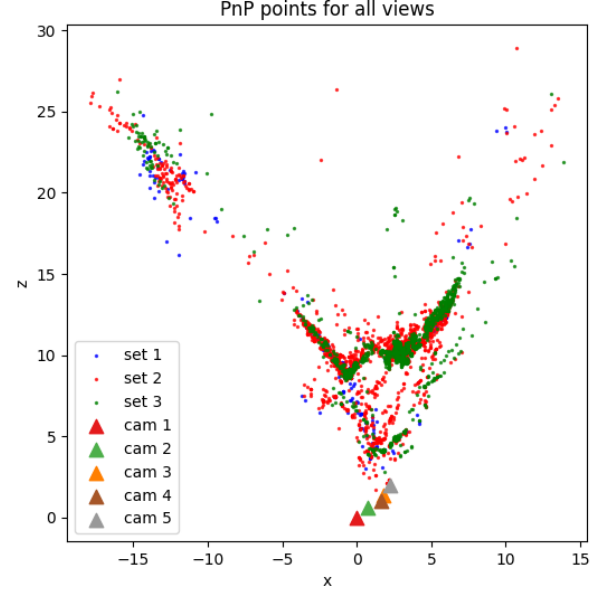


Fig. 8. World point estimates from nonlinear PnP (top-down view)

1) *Visibility Matrix*: The Visibility Matrix denotes the relationship between a point in the world and a camera pose by seeing if a point can be observed by each camera. This acts as a mask for the bundle adjustment to avoid wasting time on computations for points that are not visible to a camera, which could be a problem due to the massive size of the matrix.

2) *Bundle Adjustment*: To reduce the accumulated errors throughout the whole framework, we ran a final optimization of the world points and the camera poses. For the bundle adjustment, the camera rotations are represented with rotation vectors and packed along with translations and 3D points into a single parameter vector for optimization. The residuals are computed as difference between the observed 2D points and the projection of their 3D counterpart through each camera's projection matrix in the same way as it has been done for the nonlinear triangulation and PnP, (Equations (17)–(20) and (25)). The visibility matrix was used to make a sparsity pattern of the Jacobian so that the derivatives are only computed for the camera-point pairs that are connected (a camera-pair is connected if the world point is visible by the camera in question). Due to the large number of points in the map, this greatly improves computational efficiency.

Finally, a least squares solver simultaneously refines the camera poses and 3D points to improve the accuracy of the SfM reconstruction and minimize the errors due to compound noise in every step. A final map of all the points for the 5 images before and after the Bundle Adjustment can be seen in fig. 10. In this case, the improvement is not immensely visible, but that is not a concern for us, since the PnP results were already very accurate, and five images might not be enough

Bundle Adjustment Reprojection of 3D points for image 5



Fig. 9. Reprojected bundle adjustment points for image 5

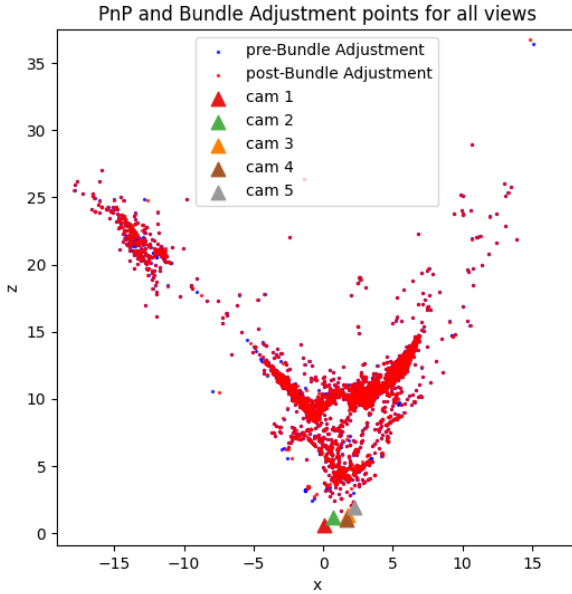


Fig. 10. World point estimates before and after bundle adjustment (top-down view)

to showcase the accumulating error of SfM. You can see the corner of the building very clearly, and the improvement throughout all of the steps is also evident as showcased in the reprojection errors presented in Table I. Refer to Figure 9 to see reprojected points for image 5. Additional results can be found in Section V.

TABLE I
MEAN REPROJECTION ERROR ACROSS ALL RELEVANT VIEWS AFTER EACH STEP OF THE PIPELINE

	Mean Reprojection Error (px)
Linear Triangulation	5.1735
Nonlinear Triangulation	2.3370
Linear PnP	17.908
Nonlinear PnP	1.785
Bundle Adjustment	0.0609

H. Putting the pipeline together

Details regarding how the previous sections all come together can be clearly seen in Algorithm 5.

Algorithm 5 High-level SfM Pipeline

Require: Image IDs (i_1, i_2) , optional additional image IDs $\{i_k\}$, intrinsics K

Ensure: Refined camera poses $\{[R_j|t_j]\}$ and 3D points $\{\mathbf{X}_m\}$

- 1: Load 2D correspondences between i_1 and i_2
- 2: Estimate F via RANSAC using $\mathbf{x}^T F \mathbf{x} = 0$
- 3: Compute essential matrix:

$$E = K^T F K$$

- 4: Extract four candidate poses $\{[R|t]\}$ from E
- 5: Fix first camera as $[I|0]$
- 6: Disambiguate second camera using cheirality condition:

$$Z_1 > 0, \quad Z_2 > 0$$

- 7: Linearly triangulate base 3D points
- 8: Refine base 3D points via nonlinear triangulation
- 9: **for** each additional image i_k **do**
- 10: Build 2D–3D correspondences with existing 3D map
- 11: Estimate pose using PnP-RANSAC
- 12: Refine pose using nonlinear PnP
- 13: Triangulate additional 3D points with existing views
- 14: **end for**
- 15: Form all projection matrices:

$$P_j = K[R_j|t_j]$$

- 16: Assemble global 3D point set $\{\mathbf{X}_m\}$
- 17: Construct visibility matrix V where

$$V_{jm} = \begin{cases} 1 & \text{if camera } j \text{ observes point } m \\ 0 & \text{otherwise} \end{cases}$$

- 18: Perform bundle adjustment:

$$\min_{\{R_j, t_j\}, \{\mathbf{X}_m\}} \sum_{j,m} V_{jm} \left\| \mathbf{x}_{jm} - \Pi\left(K[R_j|t_j]\tilde{\mathbf{X}}_m\right) \right\|_2^2$$

return Optimized poses and 3D points

III. CONCLUSION

This Phase I of this project saw a completion of 3D reconstruction from a series of 2D images with the use of a

classical method. As opposed to deep learning, which abstracts a large amount of the math behind 2D to 3D modeling, we worked through every step of the Structure from Motion framework, preventing the build up of noise through multiple layers of optimization and a final adjustment to all of the camera poses and world points to minimize the geometric error between the projected world points onto each of the image frames and the observed 2D points on the images themselves. Section V contains additional figures for point reprojection.

IV. CONTRIBUTION STATEMENT

Pau Alcolea: Function Development, Documentation and Writing
Ben Orr: Integration, Figure Visualization, Function Development, Documentation and Writing

V. APPENDIX

Linear Reprojection of 3D points for image 1

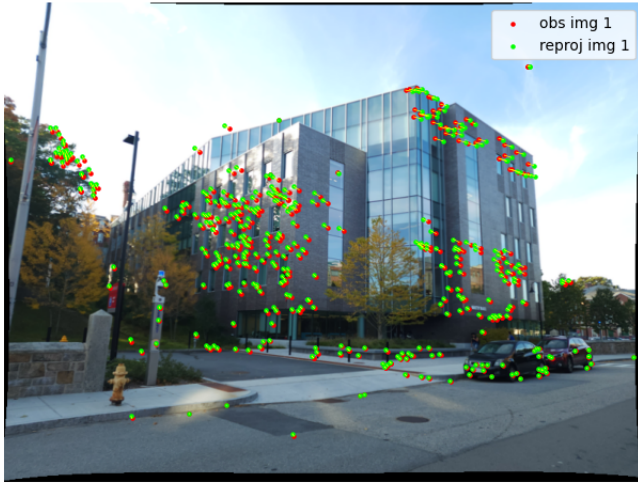


Fig. 11. Reprojected linear triangulation points for image 1

Linear PnP Reprojection of 3D points for image 3

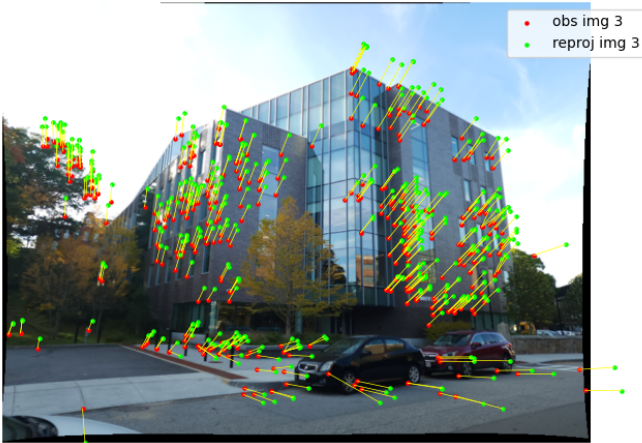


Fig. 12. Reprojected linear PnP points for image 3

Linear PnP Reprojection of 3D points for image 5

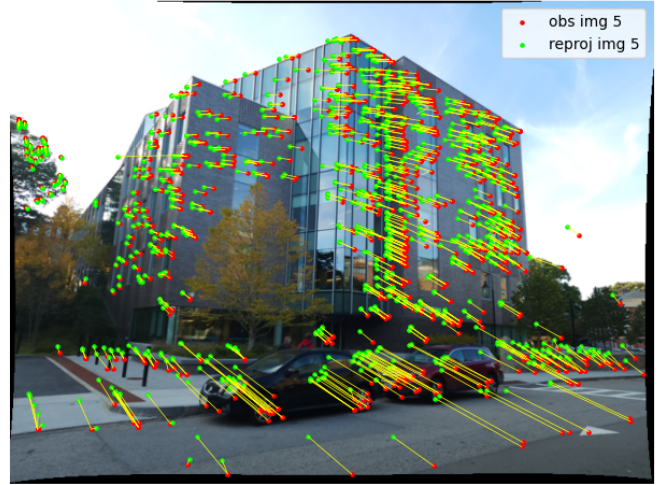


Fig. 13. Reprojected linear PnP points for image 5

Nonlinear Reprojection of 3D points for image 1

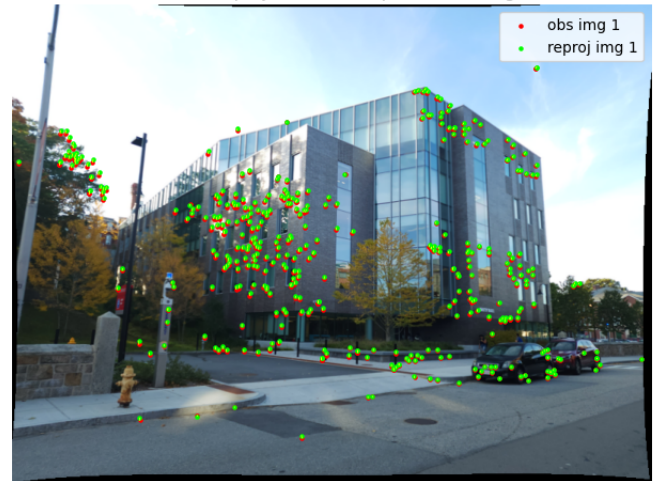


Fig. 14. Reprojected nonlinear triangulation points for image 1

Nonlinear PnP Reprojection of 3D points for image 3



Fig. 15. Reprojected nonlinear PnP points for image 3

Bundle Adjustment Reprojection of 3D points for image 1

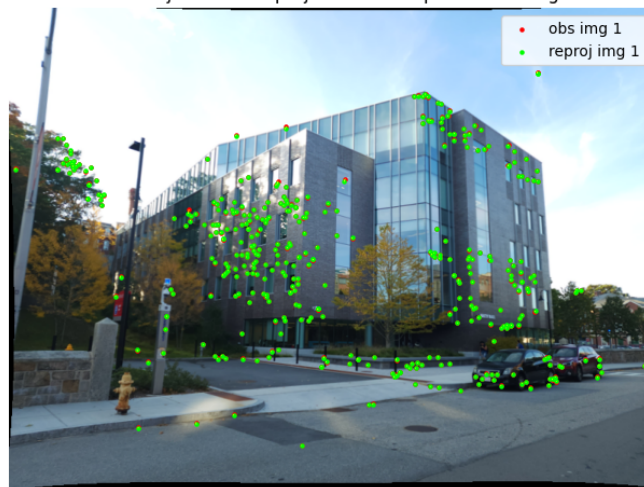


Fig. 17. Reprojected bundle adjustment points for image 1

Nonlinear PnP Reprojection of 3D points for image 5

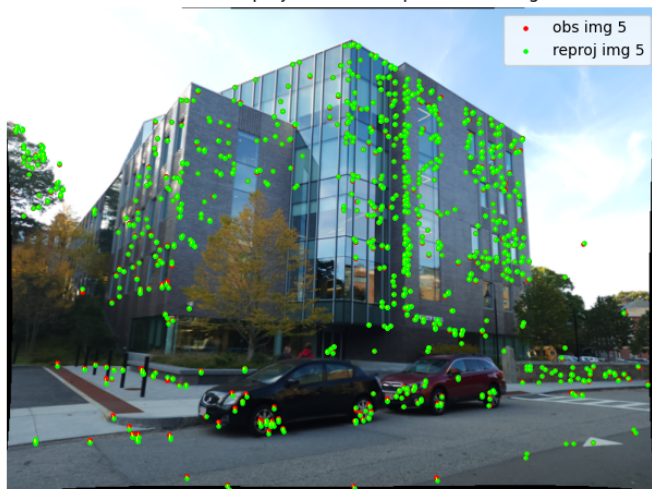


Fig. 16. Reprojected nonlinear PnP points for image 5

Bundle Adjustment Reprojection of 3D points for image 2

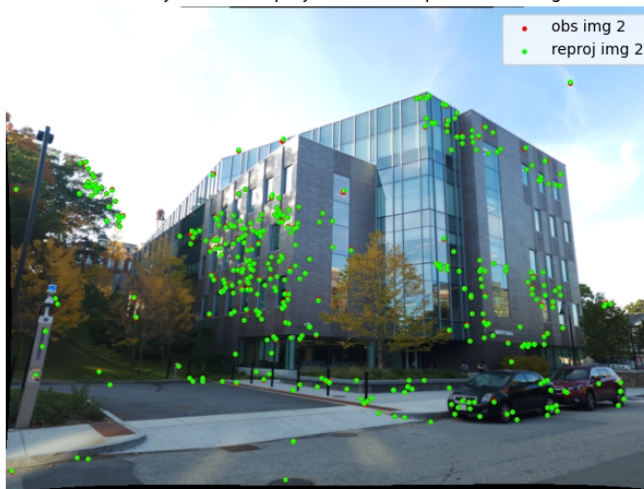


Fig. 18. Reprojected bundle adjustment points for image 2

Bundle Adjustment Reprojection of 3D points for image 3



Fig. 19. Reprojected bundle adjustment points for image 3

Bundle Adjustment Reprojection of 3D points for image 4

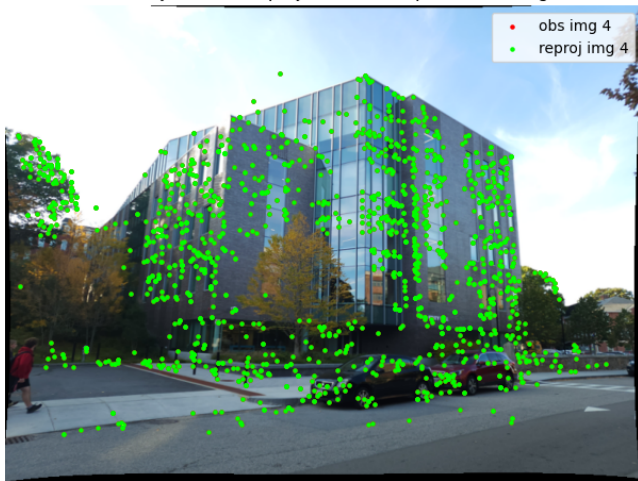


Fig. 20. Reprojected bundle adjustment points for image 4

REFERENCES

- [1] T. M. Inc., "Matlab version: 9.13.0 (r2022b)," Natick, Massachusetts, United States, 2022. [Online]. Available: <https://www.mathworks.com>
- [2] P. D. Sampson, "Fitting conic sections to "very scattered" data: An iterative refinement of the bookstein algorithm," *Computer Graphics and Image Processing*, vol. 18, no. 1, pp. 97–108, 1982. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/0146664X82901010>